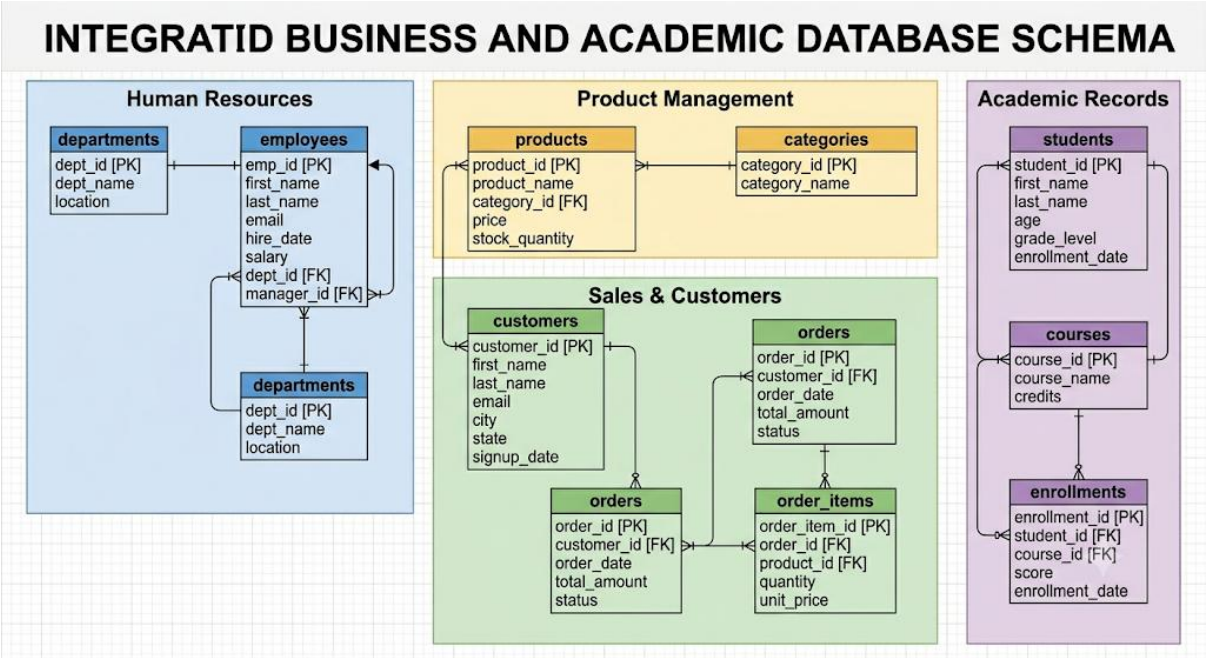
MySQL Assignment -1

All questions assume the following sample schemas:



- **employees (emp_id, first_name, last_name, email, hire_date, salary, dept_id, manager_id)**
- **departments (dept_id, dept_name, location)**
- **products (product_id, product_name, category_id, price, stock_quantity)**
- **categories (category_id, category_name)**
- **orders (order_id, customer_id, order_date, total_amount, status)**
- **order_items (order_item_id, order_id, product_id, quantity, unit_price)**
- **customers (customer_id, first_name, last_name, email, city, state, signup_date)**
- **students (student_id, first_name, last_name, age, grade_level, enrollment_date)**
- **courses (course_id, course_name, credits)**
- **enrollments (enrollment_id, student_id, course_id, score, enrollment_date)**

1. Basic SELECT and Filtering (50 Questions)

1. Write a query to retrieve all columns from the `employees` table.
2. Select only `first_name`, `last_name`, and `email` from the `employees` table.
3. Retrieve all distinct department IDs from the `employees` table.
4. Show the first name and salary of employees earning more than 60,000.
5. List employees hired after January 1, 2020.
6. Display employees whose last name is 'Smith'.
7. Find employees with a salary between 50,000 and 80,000.
8. Select employees whose first name starts with 'J'.
9. Retrieve employees whose email contains 'company.com'.
10. List employees who do not have a manager (manager_id IS NULL).
11. Show the top 5 highest-paid employees.
12. Retrieve the 10 most recently hired employees.
13. Display employees sorted by last name ascending, then first name ascending.
14. Find employees whose department ID is either 1, 2, or 3.
15. List products with a price less than 100 or stock quantity greater than 500.
16. Select customers from the city 'New York' or 'Los Angeles'.
17. Retrieve orders placed in the year 2022.
18. Show all orders with a total amount greater than 1000, sorted by total_amount descending.
19. Find products where the product name contains 'Pro' (case-insensitive).
20. List employees whose first name is exactly 5 characters long.
21. Display the concatenated full name (first_name + ' ' + last_name) as `full_name` for all employees.

Transcome Educare

22. Write a query that returns the current date and time.
23. For each product, show the product name, price, and a 10% discounted price.
24. Retrieve employees whose hire date is a Monday.
25. Select orders placed in the last 30 days.
26. Find products with NULL in the category_id column.
27. List customers who signed up between '2021-01-01' and '2021-12-31'.
28. Show distinct cities from the `customers` table.
29. Retrieve all employees ordered by salary desc, but skip the first 5 and return the next 10.
30. Display employees whose employee ID is an even number.
31. Find orders where the status is not 'Delivered'.
32. Select employees with a salary not equal to 70,000.
33. List products where the name begins with 'A' and ends with 's'.
34. Retrieve employees hired in the month of March.
35. Show the top 3 most expensive products.
36. Display employees sorted by department ID ascending, and within each department by salary descending.
37. Find all customers whose last name is NULL.
38. Select employees whose email is not from 'company.com' (i.e., does not contain it).
39. Write a query to fetch the current user and database name.
40. List all products with a stock quantity between 10 and 50 (inclusive).
41. Show orders placed on a weekend (Saturday or Sunday).
42. Retrieve distinct combinations of city and state from customers.
43. Display employees with more than 5 years of service from today.
44. Find orders where the order ID is in a given list: 1001, 1005, 1010.
45. Show the first 15% of rows from the employees table (using LIMIT with count).
46. List all columns for the 3 most recent orders.
47. Select employees whose first name has 'a' as the second letter.
48. Retrieve products that have a price ending with .99.
49. Display a list of all table names in the current database (using INFORMATION_SCHEMA).
50. Show the version of MySQL.

2. Aggregate Functions and GROUP BY (40 Questions)

51. Count the total number of employees.
52. Find the average salary of all employees.
53. Calculate the sum of all order total_amount values.
54. Retrieve the minimum and maximum product prices.
55. Count the number of distinct cities in the customers table.
56. Show the number of employees in each department.
57. For each department, display the average salary.
58. Find the total sales (sum of total_amount) per year.
59. Retrieve the highest salary in each department, along with the department ID.
60. List categories with the total number of products in each.
61. Calculate the average score per course from the enrollments table.
62. Show each customer and the total amount they have spent (sum of order totals).
63. Find departments with more than 5 employees.
64. Display products with a stock quantity that is below the average stock quantity.
65. Count how many orders each customer has placed.
66. For each product category, find the most expensive product price.
67. Show the month and total sales for each month of 2023.
68. Retrieve the employee ID and the number of direct reports (where they are the manager).
69. List departments and the number of employees hired after 2021.
70. Find courses with at least 10 enrollments.
71. Display the average salary by department, but only for departments where the average is above 60,000.
72. Show the total quantity sold for each product (from order_items).
73. Calculate the variance and standard deviation of salaries across the company.
74. For each department, find the difference between the highest and lowest salary.
75. List the top 3 customers by total order amount.
76. Display the number of orders per status (e.g., 'Pending', 'Shipped', 'Delivered').

Transcribe Educare Pvt. Ltd.

77. Show the department ID and the median salary (using PERCENTILE_CONT if MySQL 8+; otherwise suggest using a subquery approach) – Write a query to find the median salary per department (hint: using GROUP BY and window functions).
78. Find employees who earn more than the average salary of their own department.
79. Count the number of products that have never been ordered (using LEFT JOIN and GROUP BY).
80. Calculate the running total of sales per month (you may use window function here, but if placed in aggregate section, suggest using variables; for assignment, just ask: "Calculate monthly sales and the cumulative sum across months.").
81. Show the highest single-day order total for each customer.
82. Retrieve products that are priced above the average price of their category.
83. For each city, find the number of customers and the average order amount per customer.
84. List the department with the highest total salary expenditure.
85. Display the percentage of employees in each department relative to the total company employees.
86. Find the average quantity of items per order.
87. Show the year of hire and the count of employees hired in that year, only for years with more than 2 hires.
88. Calculate the total revenue generated per product category.
89. Retrieve the employee ID and their rank by salary within the company (use window function here if allowed; can be part of aggregate section as challenge). (If strictly aggregates, ask: "Use a derived table to rank employees by salary without window functions.")
90. Find the department(s) with the maximum number of employees.

3. Joins (50 Questions)

91. Write an INNER JOIN query to list employees with their department names.
92. Show all employees and their department names, including employees without a department.
93. Display all departments, even those with no employees.
94. Perform a CROSS JOIN between a small table of letters ('A','B','C') and numbers (1,2,3) to generate all combinations.
95. List product names and their category names.
96. Retrieve all orders with customer full name, city, and order date.
97. Show order items along with the product name and the order ID.
98. Find all employees and their managers (self-join on manager_id).
99. List pairs of employees who work in the same department (exclude pairing an employee with themselves).
100. Display customers who have never placed an order.
101. Show products that have never been ordered.
102. For each employee, display their name and the name of their manager; if no manager, show 'Top Manager'.
103. List all students and the courses they are enrolled in, including students with no enrollments.
104. Retrieve courses and the number of enrolled students, even courses with zero students (use LEFT JOIN and GROUP BY).
105. Find all possible combinations of colors ('Red','Green','Blue') and sizes ('S','M','L') using CROSS JOIN.
106. Write a query to join three tables: orders, order_items, and products, to show order ID, product name, and quantity.
107. Retrieve the names of employees who work in the same department as 'John Doe'.
108. List employees who earn more than their manager.
109. Show all orders placed by customers from 'California', along with the order total.
110. Display the top 5 products by total sales amount (using joins and aggregation).
111. Find employees who have the same hire date as another employee (self-join, different emp_id).
112. List the manager names and the count of employees reporting to them (zero if none).
113. Show a full outer join simulation (LEFT JOIN UNION RIGHT JOIN) between two small tables.
114. Retrieve all employees and their department location.
115. Display products and their category, but only categories that have at least 1 product (INNER JOIN).
116. Use a NATURAL JOIN (if applicable) to join employees and departments on dept_id.
117. Join the orders table with a subquery that computes the average order amount per customer, and show orders above that average.
118. List customers and their most recent order date (using LEFT JOIN with a subquery).
119. Find all employees who do not manage anyone (their emp_id not in manager_id list) using LEFT JOIN.
120. For each product, show the name and the total number of units sold, using a join with order_items.
121. Retrieve employees whose manager is in a different department.
122. Show a matrix of employee-manager relationships using a self-join.

Transcome Educare Pvt. Ltd.

123. List all orders, the customer name, and the sum of item quantities per order.
124. Find customers who placed orders in all months of 2023 (harder: join with month series).
125. Write a query to join the employees table with itself to find pairs of employees hired within 30 days of each other.
126. Display products that belong to categories that have more than 10 products.
127. Show the cartesian product of a set of priorities ('High','Medium','Low') and a set of statuses ('Open','Closed') using CROSS JOIN, and assign a dummy ID.
128. Join employees with departments using USING clause.
129. Retrieve employees along with the number of orders they have processed (if there's a salesperson_id in orders, assume it links to employees).
130. Use a LEFT JOIN to find customers with no orders and their signup date.
131. List all possible supervisor-subordinate pairs from a flat table that only has emp_id and supervisor_id (recursive structure handled later; for now just direct).
132. Find products whose price is higher than the average price of products in the same category (using a join with derived table).
133. Show each order and the running count of items in that order (join order_items and use row numbering later; here just join).
134. Demonstrate a join with an ON clause that includes multiple conditions (e.g., not only key match but also date range).
135. Join employees table with a static range table (e.g., salary brackets) to categorize each employee.
136. Write a query that joins a table to itself twice (e.g., employee -> manager -> senior manager).
137. Retrieve customers and their latest order details (customer name, order ID, order date) using a join with a subquery that finds max order date per customer.
138. List employees and whether they are a manager or not (flag) using LEFT JOIN and CASE.
139. Show order items with product name and also the category name (three-table join).
140. Display employees and the total revenue they generated (if salesperson), also include those with no sales.

4. Subqueries (40 Questions)

141. Find employees whose salary is greater than the overall average salary.
142. Select the names of employees who work in the 'Sales' department (subquery to get dept_id).
143. Retrieve products that have a price equal to the maximum price in the products table.
144. List the employee with the highest salary in each department using a correlated subquery.
145. Find customers who have placed orders with a total amount greater than 500 (using EXISTS).
146. Show departments that have no employees (NOT EXISTS).
147. Display employees who earn more than the average salary of any department (use > ANY).
148. Retrieve products that are more expensive than all products in category 1 (use > ALL).
149. Find employees whose manager's salary is less than theirs (correlated subquery).
150. List the top 5 most recent orders per customer using a subquery.
151. Use a scalar subquery to display each employee's salary and the overall average salary in the same result set.
152. Show products whose stock quantity is below the average stock quantity of their category.
153. Retrieve orders where the total_amount is greater than the customer's average order amount.
154. Find the department with the highest total salary (using a subquery in FROM).
155. List all employees who do not have any subordinate (NOT EXISTS on manager_id).
156. Display each product category and the product with the maximum price, using a subquery.
157. Show customers who have never returned an item (assuming a returns table; subquery with NOT IN).
158. Retrieve employees whose salary is in the top 3 salaries within their department (subquery with LIMIT in a correlated way, or window function; here ask: "Find employees whose salary is in the top 3 of their department using a correlated subquery that counts how many have higher salary").
159. List orders that contain more than the average number of items per order (subquery to compute average count).
160. Find products that appear more than once in order_items on the same order (subquery with HAVING COUNT(*) > 1).
161. Use a subquery in the SELECT clause to add a column showing the number of employees in each employee's department.
162. Select departments where the minimum salary is greater than the maximum salary of department 5.
163. Retrieve the second highest salary using a subquery.
164. Display all employees who were hired on the same date as the first employee hired.
165. Show products with a price that is not within 10% of the average price.
166. Find customers whose total spending is above the average total spending of all customers.
167. List employees whose email domain is the most common domain in the company (subquery to find most frequent domain).

Transcore Educare Pvt. Ltd.

168. Retrieve the order with the highest number of distinct products.
169. Use a correlated subquery to find employees whose salary is less than the average salary of employees hired in the same year.
170. Show the employee with the longest tenure (earliest hire_date).
171. List all orders and a flag indicating if the order total exceeds the customer's lifetime average order value (correlated subquery).
172. Find departments that have at least one employee earning more than 100,000.
173. Retrieve all products whose price is greater than the price of any product in category 2.
174. Use a subquery to create a derived table that calculates the total sales per category, then join to categories to show category name and total.
175. Find the employee(s) with the 5th highest salary (without LIMIT/OFFSET, using subquery).
176. Show customers who placed orders in every month of 2023 (subquery to get distinct months count).
177. List employees working in departments located in 'New York' (nested subquery).
178. Retrieve employees and their salary rank within the company using a subquery that counts distinct higher salaries.
179. Display the product that has the largest total quantity sold, using a subquery to compute aggregate.
180. Find the department ID and name where the average salary is higher than the company-wide average.

5. Set Operations (30 Questions)

181. (Mysql supports UNION, UNION ALL; INTERSECT and EXCEPT can be simulated. Questions will use UNION, UNION ALL, and simulate INTERSECT/MINUS.)
182. Combine the list of customer cities and employee cities into a single distinct list.
183. Union two queries that select product names from different categories (e.g., 'Electronics' and 'Books') and show all.
184. Merge the first and last names of employees and customers into one column of names (union all).
185. Use UNION ALL to combine order IDs from orders in 2022 and 2023 and keep duplicates.
186. Find employees who are also customers (same email) – use INTERSECT simulation (INNER JOIN or EXISTS).
187. Retrieve products that are in category 1 but not in category 2 (simulate EXCEPT with NOT IN).
188. Show the distinct combination of department ID and location from both departments and a legacy_departments table (UNION).
189. Combine the top 5 expensive products and top 5 cheapest products into a single result set (UNION).
190. Write a query to mimic INTERSECT between two sets: employees in department 1 and employees earning > 70000.
191. Mimic EXCEPT: list customers who have placed orders but have never made a return (if returns table).
192. Union two queries that count employees and customers by city, showing city and a type indicator.
193. Use UNION to create a report that shows 'Employee' as source, emp_id as ID, name for employees, and similarly 'Customer' for customers.
194. Find products that are either out of stock (quantity=0) or have never been ordered (using UNION of two distinct sets).
195. Show all distinct email domains from employees and customers (UNION).
196. Combine the results of three queries: active orders, pending orders, cancelled orders into one list with status column.
197. Demonstrate UNION with ordering: select top 5 employees by salary and top 5 by hire_date, sorted globally by name.
198. Retrieve IDs that exist in both a temp_table and products table (INTERSECT using IN).
199. Find customer IDs that are in the customers table but not in the blacklist table (EXCEPT using NOT IN).
200. Merge the results of monthly sales for 2022 and 2023 into a single dataset for comparison (UNION ALL with year column).
201. Create a full list of all dates from an orders table and a returns table (UNION of order_date and return_date).
202. Simulate EXCEPT using LEFT JOIN IS NULL: get employees who are not managers.
203. Union three different product lines (by category) and add a constant column indicating the line.
204. Show the union of all first names from employees and customers, but with a count of occurrences (group after UNION ALL).
205. Use UNION to consolidate multiple tables with similar structure (e.g., sales2019, sales2020) into one result.
206. Retrieve a list of IDs that are in table A or table B but not both (symmetric difference) using UNION of EXCEPTs.
207. Find products that have had a price increase and also a price decrease (from a price_changes table) using UNION of two conditions.
208. Create a query that uses UNION to generate a sequence of numbers 1...10.
209. Combine employee names with a literal 'No Manager' row as a UNION.
210. List all order statuses from orders and also from archived_orders, with duplicates removed.

Transcome Edware Pvt. Ltd.

211. Show distinct product_id values that appear either in order_items or in wishlist_items using UNION.

6. Data Modification (DML) (30 Questions)

212. Insert a new employee into the employees table with specified values.
213. Insert multiple rows into the products table in a single statement.
214. Update the salary of all employees in department 3 by 10%.
215. Delete all orders placed before 2020.
216. Use INSERT ... SELECT to copy customers from a temporary table into the main customers table.
217. Update the stock quantity of a product by subtracting the quantity ordered from a specific order.
218. Set the manager_id of an employee to NULL when they become top-level.
219. Delete customers who haven't placed any orders (using a subquery or join).
220. Insert into a table only if a record with the same unique key doesn't exist (INSERT IGNORE or ON DUPLICATE KEY UPDATE).
221. Use ON DUPLICATE KEY UPDATE to increment a counter in a daily_sales_summary table.
222. Update multiple columns in the employees table (e.g., salary and department) based on a condition.
223. Delete all records from a log table older than 30 days.
224. Insert a new order and related order_items using a transaction (explain transaction steps).
225. Update the price of all products in a category by multiplying by 1.05, then log the changes in a price_change_log table (use a trigger or manual? For DML, just do update and then insert into log using a subquery).
226. Write a DELETE statement that removes duplicate rows from a table based on a subset of columns, keeping the one with the lowest ID.
227. Replace (REPLACE INTO) an employee record entirely based on emp_id.
228. Use an UPDATE with a JOIN to set the department name in a denormalized column for reporting.
229. Insert 100 random rows into a test_numbers table using a recursive CTE or INSERT ... SELECT with a series generator.
230. Update the total_amount of an order by summing its order_items' quantity*unit_price.
231. Perform a bulk update by setting the status to 'Archived' for all orders older than 5 years.
232. Use a correlated update to set the manager's salary by adding a bonus column from a managers table.
233. Delete from a table where a foreign key constraint might fail; use queries to delete child rows first.
234. Insert a new product and automatically set its creation timestamp using DEFAULT.
235. Write a multi-table DELETE to remove customers and their related orders when the customer is deleted.
236. Use INSERT INTO ... VALUES with a subquery that returns a single computed value.
237. Update employee emails to a new format: firstname.lastname@newcompany.com.
238. Delete all rows from a table and reset auto-increment (TRUNCATE).
239. Simulate an upsert operation by attempting to update, and if no row affected, then insert (use a stored procedure or conditional logic; for DML, just mention technique).
240. Copy selected columns from one table to a new table (CREATE TABLE ... AS SELECT).
241. Set the salary of the lowest-paid employee in each department to the department's average salary.

7. DDL and Constraints (40 Questions)

242. Create a new table `projects` with columns: project_id INT PRIMARY KEY AUTO_INCREMENT, project_name VARCHAR(100) NOT NULL, start_date DATE, end_date DATE, budget DECIMAL(10,2).
243. Add a column `phone` VARCHAR(15) to the employees table.
244. Rename the table `clients` to `customers`.
245. Drop the column `obsolete_column` from a table.
246. Define a composite primary key on `order_id` and `product_id` for the order_items table.
247. Add a foreign key constraint on `dept_id` in employees referencing dept_id in departments.
248. Create a UNIQUE constraint on the `email` column of employees.
249. Add a CHECK constraint to ensure salary is greater than 0.
250. Set a default value of 'Active' for the status column in orders.
251. Alter a column to make it NOT NULL.
252. Remove a foreign key constraint.
253. Create an index on `last_name` in employees.
254. Create a full-text index on `product_name` and `description`.
255. Add a new column `updated_at` TIMESTAMP with automatic update on row modification.
256. Use CREATE TABLE LIKE to create an empty copy of the employees table structure.
257. Create a temporary table for session-specific data.
258. Define a table with an ENUM column for status values ('Pending','Processing','Completed').
259. Create a table with a JSON column to store extra attributes.

Tranzcomm Edu Care Pvt. Ltd.

260. Add a constraint to restrict the allowed values of a column using CHECK (e.g., price >= 0).
261. Drop a table if it exists.
262. Truncate a table (DDL) and explain difference from DELETE.
263. Create a new schema/database.
264. Use ALTER to change the data type of a column from INT to BIGINT.
265. Set the character set and collation for a table.
266. Create a view that selects active employees.
267. Drop a view if it exists.
268. Rename a database (if allowed) or just rename a table.
269. Add a COMMENT to a table and a column.
270. Disable foreign key checks temporarily for bulk loading.
271. Create a table with a generated column that concatenates first_name and last_name.
272. Add a primary key to an existing table without one.
273. Use SHOW CREATE TABLE to retrieve the DDL of a table.
274. Create a sequence or use AUTO_INCREMENT with custom starting value.
275. Add a column with a specific position (FIRST or AFTER another column).
276. Drop an index.
277. Alter a table to add a UNIQUE composite constraint on (first_name, last_name).
278. Change the storage engine of a table to InnoDB.
279. Create a table that inherits structure from another but with additional columns (CREATE TABLE ... AS SELECT with 0 rows).
280. Add a foreign key with ON DELETE CASCADE.
281. Drop and recreate a primary key.

8. Views (20 Questions)

282. Create a view that shows employee full name, department name, and salary.
283. Create a view `high_earners` that includes employees with salary > 100,000.
284. Write a view that aggregates total sales per customer.
285. Create a view with check option to ensure inserted rows meet a condition.
286. Alter an existing view to add a new column.
287. Drop a view.
288. Show the definition of a view (SHOW CREATE VIEW).
289. Use a view to simplify a complex join of orders, customers, and order_items.
290. Query a view that includes a window function (e.g., rank employees by salary).
291. Insert data through a view into a base table (updatable view conditions).
292. Discuss when a view is not updatable and provide an example of a non-updatable view.
293. Create a view that uses a UNION.
294. Use a view to restrict access to sensitive columns (e.g., hide salary from some users).
295. Create a materialized view (simulate using a table and periodic refresh).
296. Write a view that calculates the running total of sales per day.
297. Use WITH LOCAL CHECK OPTION in a view hierarchy.
298. Create a view that shows top N products by sales, where N is a parameter (parameterized view using a function or session variable).
299. List all views in the current database from INFORMATION_SCHEMA.
300. Create a recursive view (MySQL doesn't support recursive views directly; assignment: "Discuss how to simulate a recursive view using a CTE in a stored procedure.").
301. Test if a view exists before creating it (CREATE OR REPLACE VIEW).

9. Indexes (15 Questions)

302. Explain the difference between a clustered and secondary index in InnoDB.
303. Create a composite index on (dept_id, hire_date).
304. Use EXPLAIN to analyze a query and identify if an index is used.
305. Drop a redundant index.
306. Create a unique index on the email column.
307. How to create a prefix index on a TEXT column?
308. Show all indexes on a table using SHOW INDEX.
309. Create a descending index on order_date (MySQL 8+).
310. Discuss the impact of too many indexes on write performance.
311. Write a query that forces the use of a specific index using FORCE INDEX.

Transcripe Educare Pvt. Ltd.

312. Create a spatial index on a geometry column.
313. Identify unused indexes using the sys.schema_unused_indexes view.
314. Rebuild or optimize an index (ALTER TABLE ... ENGINE=InnoDB).
315. Create a full-text index and a query using MATCH AGAINST.
316. Use invisible indexes (MySQL 8+) to test performance before dropping.

10. Stored Procedures and Functions (30 Questions)

317. Write a stored procedure that takes a department ID and returns the number of employees.
318. Create a procedure that gives a raise to all employees in a given department by a specified percentage.
319. Write a function that calculates the age of an employee given their birth date.
320. Create a procedure that inserts a new order and all its items, handling rollback on error.
321. Write a function that returns the full name of an employee given their ID.
322. Develop a procedure that outputs the top N customers by total sales.
323. Use a cursor inside a procedure to iterate over a result set and perform updates.
324. Create a function that determines the discount category based on total purchases (e.g., >10000 'Gold').
325. Write a procedure that archives old orders (move to archive table) and deletes from main.
326. Create a function that calculates the factorial of a number iteratively.
327. Write a procedure with an OUT parameter that returns the average salary of a department.
328. Use a WHILE loop in a procedure to insert a sequence of dates into a table.
329. Create a function that validates an email format using REGEXP.
330. Develop a procedure that generates a sales report between two dates and inserts into a report table.
331. Write a function that splits a comma-separated string and returns the number of elements.
332. Create a procedure with conditional logic to update salary based on performance rating.
333. Use SIGNAL to raise a custom error in a procedure if invalid input.
334. Write a procedure that uses a transaction to transfer funds between two accounts (custom table).
335. Create a recursive stored procedure to traverse a tree (e.g., employee hierarchy).
336. Write a function that returns the nth highest salary (n as parameter).
337. Create a procedure that uses PREPARE and EXECUTE to run a dynamic SQL query.
338. Write a function that computes the distance between two zip codes (assuming a formula).
339. Develop a procedure that bulk inserts random test data.
340. Create a function that masks an email (e.g., j***@domain.com).
341. Write a procedure that synchronizes two tables, inserting new records and updating existing ones.
342. Use GET DIAGNOSTICS to capture the number of rows affected by the previous statement.
343. Create a stored procedure that returns multiple result sets.
344. Write a function that converts a datetime to a human-readable "time ago" string.
345. Create a procedure that applies a complex business logic: reorder products that fall below stock threshold by inserting into a purchase_orders table.
346. Build a logging framework using a procedure that writes to a log table with timestamp.

11. Triggers and Events (15 Questions)

347. Create a BEFORE INSERT trigger that automatically sets the `created_at` timestamp.
348. Write an AFTER UPDATE trigger that logs changes to employee salary into an audit table.
349. Create a trigger that prevents deletion of a customer if they have active orders.
350. Build a BEFORE DELETE trigger that archives the row into a backup table.
351. Write an AFTER INSERT trigger on order_items that decreases product stock.
352. Create a trigger that validates that an order total is positive before insert/update.
353. Develop a trigger that updates a `last_modified` column on the parent table when a child row is modified.
354. Use a trigger to maintain a denormalized total_amount in the orders table whenever order_items are inserted, updated, or deleted.
355. Create an event that runs daily to delete expired sessions.
356. Write an event that aggregates daily sales and inserts into a summary table.
357. Schedule an event that runs every hour to optimize/reindex a table.
358. Create a trigger that automatically assigns a customer to a sales representative based on region.
359. Build a trigger that prevents a salary decrease beyond 10% on update.
360. Use SIGNAL in a trigger to abort an operation with a custom error message.
361. Create an event that executes a stored procedure every Monday at 8 AM.

12. Transactions and Locking (15 Questions)

Transcom Software Pvt. Ltd.

362. Explain ACID properties and how MySQL InnoDB ensures them.
363. Write a transaction that transfers money between accounts, with COMMIT and ROLLBACK.
364. Use SAVEPOINT within a transaction to roll back to a specific point.
365. Demonstrate a deadlock scenario with two sessions and explain how to resolve.
366. Set transaction isolation level to REPEATABLE READ and show an example avoiding phantom reads.
367. Use SELECT ... FOR UPDATE to lock rows for a subsequent update.
368. Create a transaction that reads the same row twice and demonstrates non-repeatable read if isolation is READ COMMITTED.
369. Query to show current transaction isolation level.
370. Use LOCK TABLES to prevent writes while performing a bulk read.
371. Explain optimistic vs pessimistic locking with example (version column).
372. Write a query with SKIP LOCKED (MySQL 8+) to process jobs from a queue without waiting.
373. Demonstrate a transaction that uses NOWAIT to fail immediately if a row is locked.
374. Show how to set autocommit off and manually control commits.
375. Create a stored procedure that handles deadlock retry logic.
376. Use SHOW ENGINE INNODB STATUS to analyze lock waits.

13. Window Functions (50 Questions)

377. Assign row numbers to employees ordered by salary descending.
378. Rank employees by salary within each department (use RANK()).
379. Use DENSE_RANK() to rank products by price within each category.
380. Show the difference in salary from the previous employee when ordered by hire_date (LAG).
381. Display the next order date for each customer using LEAD().
382. Calculate the running total of sales amount over order date.
383. Compute a moving average of the last 3 orders' totals for each customer.
384. Partition employees by department and assign a row number resetting per department.
385. Find the first value (earliest hire date) in each department using FIRST_VALUE().
386. Get the last order date per customer using LAST_VALUE() with RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.
387. Use NTILE(4) to divide employees into quartiles based on salary.
388. Calculate the percentage of total salary that each employee earns within their department (ratio_to_report simulation using SUM).
389. For each order, compute the difference between its total and the average total of the last 5 orders.
390. Show cumulative distribution of salaries using CUME_DIST().
391. Use PERCENT_RANK() to find percentile rank of each product's price.
392. Write a query that numbers rows in the result set without ordering (ORDER BY (SELECT NULL)).
393. Find the highest salary in each department using MAX() as a window function.
394. Display each sale along with the running count of sales per product.
395. Compare each employee's salary with the department average using a window aggregate.
396. Assign a unique sequential ID to orders per customer ordered by date.
397. Show the year-over-year growth of sales using LAG on yearly totals.
398. Determine the median salary per department using PERCENTILE_DISC or simulation with window functions.
399. Use ROW_NUMBER() to delete duplicate rows keeping the first occurrence.
400. Compute the sum of a column over a rolling 7-day window (RANGE INTERVAL).
401. Identify islands of consecutive dates in attendance data using ROW_NUMBER() difference technique.
402. Rank orders by amount within each month.
403. Retrieve the second highest salary in each department using DENSE_RANK and a subquery.
404. Show each transaction and the balance as a running total.
405. Find the average price of products within the same category and compare each product's price to it.
406. Use LEAD to compute the time between consecutive orders for each customer.
407. Partition by customer and order by date, number the orders, then filter to just the first 3 orders per customer.
408. For each department, show the employee with the highest salary, and the difference to the next highest (using LEAD/LAG after ranking).
409. Compute a weighted moving average (e.g., last 3 with weights 0.5, 0.3, 0.2) using window functions and manual calculation.
410. Create a "gaps and islands" solution for identifying consecutive login streaks for users.
411. Show each employee's salary and the cumulative distribution of salaries across the whole company.
412. Use ROW_NUMBER() to paginate results (fetch rows 21-40).
413. Find the top N records per group using ROW_NUMBER() (e.g., top 2 products per category by sales).
414. Show the running total of quantity sold per product over time.

Transcribe Edware Pvt. Ltd.

415. For each order, include the previous and next order dates of the same customer.
416. Compute the mode (most frequent value) of a column using window functions (count and rank).
417. Identify the point where cumulative sales exceed a threshold.
418. Use FIRST_VALUE() with a filter to get the first non-null next value.
419. Compare current month sales with previous month using LAG on a monthly summary CTE.
420. Use window functions to fill down the last non-null value in a column (e.g., carry forward last known price).
421. Display the percentage of orders that are each customer's orders, using COUNT(*) OVER.
422. Find employees whose salary is above the 90th percentile within their department (NTILE or PERCENT_RANK).
423. Assign a unique "batch number" to every 10 rows.
424. Use CUME_DIST to find the top 20% of customers by total spending.
425. Implement a query that returns the difference from the overall average salary without subquery (using window AVG).
426. For each employee, calculate the difference between their salary and the average salary of the department they belong to.

14. Common Table Expressions (Non-Recursive) (30 Questions)

427. Write a CTE to list employees with salary > 80000, then select from it.
428. Create a CTE that defines the top 5 products by sales and join with products table.
429. Use multiple CTEs in a single query to break down a complex query (e.g., order_summary, customer_summary, then join).
430. Define a CTE that calculates average salary per department, then use it to list employees above their dept average.
431. Write a CTE that simulates a temporary view for a monthly sales report.
432. Use a CTE with a window function to rank employees, then filter to rank <= 3.
433. Chain CTEs: first CTE computes totals, second CTE adds rankings, final select filters.
434. Write a recursive CTE? (No, keep non-recursive; for this section, just use a CTE to generate series via UNION but not recursive. For example, a series of numbers using row_number() from an existing large table.)
435. Create a CTE that pivots sales data by year (conditional aggregation in CTE, then select).
436. Use a CTE to delete duplicate rows (with ROW_NUMBER) and then DELETE using the CTE (if updatable).
437. Build a CTE that computes the year-to-date sales.
438. Use a CTE to isolate expensive products and then find the total stock value of those products.
439. Create a CTE that calculates the median price per category using PERCENTILE_CONT or NTILE.
440. Write a query with a CTE that returns the most recent order for each customer.
441. Define a CTE that holds a list of blacklisted customer IDs, then exclude them from a sales report.
442. Use a CTE as a source for an INSERT statement (INSERT INTO ... WITH ...).
443. Create a CTE that generates a calendar table for the current year (using series generation without recursion: using UNION of fixed quarters maybe). And then use it to fill missing dates in a sales report.
444. Refactor a correlated subquery into a CTE for better readability.
445. Use a CTE to store the result of a GROUP BY and then apply another GROUP BY on top (resume aggregation).
446. Combine two CTEs with UNION.
447. Use a CTE in a CREATE VIEW statement (CREATE VIEW AS WITH ...).
448. Write a CTE that adds row numbers to all employees, then self-join the CTE to pair employees with adjacent row numbers.
449. Create a CTE to split a comma-separated string into rows using JSON_TABLE or a stored function; then use it in a query.
450. Use CTEs to implement a step-by-step data cleaning pipeline: trim whitespace, standardize dates, remove invalid rows.
451. Benchmark: take a complex subquery and rewrite it using CTEs; provide both versions (assignment question: "Rewrite the following subquery using a CTE...").
452. Write a CTE that uses a window function and is referenced twice in the final query.
453. Create a CTE to compute the running total of orders per day.
454. Use CTE to isolate groups with HAVING and then join back to detail.
455. Demonstrate how CTEs can improve readability by naming intermediate results.
456. Write a CTE that prepares data for a chart (e.g., cohort analysis).

15. Recursive CTEs (20 Questions)

457. Generate a series of numbers from 1 to 100 using a recursive CTE.

Transcreme Edutech Pvt. Ltd.

458. Create a recursive CTE to traverse an employee hierarchy (manager tree) and show the path from top manager to each employee.
459. Write a recursive CTE that calculates the factorial of 10.
460. Generate a date sequence for the current month, one row per day.
461. Find all subordinates of a given manager down multiple levels.
462. Compute the total salary budget for a department including all sub-departments in a hierarchical org structure.
463. Use a recursive CTE to explode a bill of materials (BOM) table (component, parent) to list all components of a product.
464. Create a recursive query to solve the "Eight Queens" puzzle? (Too complex, maybe just: "Write a recursive CTE to generate all combinations of two letters A,B,C").
465. Traverse a category tree (parent category) to build a full path like 'Electronics/Computers/Laptops'.
466. Use recursion to perform a graph traversal (e.g., find all connected nodes in a network).
467. Generate a Fibonacci sequence up to 100 using a recursive CTE.
468. Write a CTE that calculates the level of each node in a tree and builds an indented list.
469. Find all descendants of a particular category, including the category itself.
470. Create a recursive CTE that calculates the running total without window functions (using recursion with carry forward).
471. Detect cycles in a hierarchical table using recursion with a cycle detection column.
472. Use a recursive CTE to split a comma-separated string into rows (alternative to numbers table).
473. Write a recursive query that returns the transitive closure of a "friendship" table.
474. Generate all months between two dates using a recursive CTE.
475. Implement a recursive query to perform a division operation as repeated subtraction (integer division).
476. Create a recursive CTE that simulates the expansion of a grouped bill (e.g., each level adds tax).

16. Advanced Topics: Pivot, JSON, Dynamic SQL, Performance (25 Questions)

477. Write a query to pivot the sum of sales per year for each product category (static pivot using CASE).
478. Use the JSON functions to extract a value from a JSON column.
479. Create a query that returns a JSON array of all employees in a department.
480. Use JSON_TABLE to convert a JSON array into relational rows.
481. Write a query that produces a JSON object containing department name and employee count.
482. Update a specific key in a JSON document using JSON_SET.
483. Use dynamic SQL to create a pivot where the column values are not known in advance.
484. Write a prepared statement that accepts a table name as parameter and returns its row count.
485. Use EXPLAIN ANALYZE (MySQL 8.0.18+) to analyze a slow query.
486. Profile a query using SHOW PROFILES (if enabled) and identify bottlenecks.
487. Optimize a query by adding appropriate indexes; measure with EXPLAIN.
488. Use the optimizer hints (e.g., JOIN_ORDER, INDEX) to influence execution plan.
489. Write a query that identifies missing indexes using the sys.schema_unused_indexes or performance_schema.
490. Use window functions to create a cohort retention analysis (users by signup month and activity month).
491. Write a query that calculates the distance between geographic points using ST_Distance_Sphere.
492. Use FULLTEXT search with MATCH and relevance ranking.
493. Create a generated column that extracts a year from a DATE column.
494. Use a recursive CTE combined with JSON_ARRAYAGG to build a hierarchical JSON output.
495. Write a query that uses a lateral derived table (MySQL 8.0.14+) to join a subquery that references columns from preceding tables.
496. Use the RANK() function in a subquery to answer "How many customers have total sales in the top 10%?".
497. Write a query that combines INTERSECT and EXCEPT with CTEs for a complex set operation.
498. Implement a queue using SKIP LOCKED for processing items without overlap.
499. Write a query that uses a window frame with RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW.
500. Use the REGEXP_REPLACE function to clean phone numbers.
501. Final challenge: Design a complete sales dashboard query that uses CTEs, window functions, JSON aggregation, and conditional pivots to output a summary of daily sales, top products, and percent change from previous day, all in a single query.

Transcribe Edtech Pvt. Ltd.